

Deep Learning

Neural Networks, Architectures, and How They Are Trained

This guide explains how neural networks work from the ground up, walks through the architectures that power modern computer vision, language, and sequence modeling, and covers the training techniques that make it possible to fit models with millions or billions of parameters. It builds directly on classical machine learning concepts.

1. From Machine Learning to Deep Learning

Deep learning is a subfield of machine learning built around neural networks with many layers — the "deep" in the name refers to that layer depth. What makes neural networks distinct from classical models is that they learn their own feature representations directly from raw data, rather than relying on hand-engineered features. This is why deep learning has been especially transformative in domains like images, audio, and text, where manually designing good features is difficult.

The trade-off is data and compute: deep networks generally need far more training examples and far more computation than classical models to reach strong performance, which is why deep learning's rise tracked closely with the availability of GPUs and large datasets.

2. The Building Blocks of a Neural Network

2.1 Neurons, Layers, and Activation Functions

A single artificial neuron computes a weighted sum of its inputs, adds a bias term, and passes the result through a nonlinear activation function. Stacking neurons into layers, and stacking layers on top of each other, is what gives a network the capacity to approximate very complex functions — a result formalized by the universal approximation theorem. Without a nonlinear activation function, no matter how many layers are stacked, the whole network would collapse to a single linear transformation.

- ReLU ($\max(0, x)$) — the default choice in most hidden layers; simple and fast to compute.
- Sigmoid — squashes output to $(0, 1)$; used for binary classification outputs.
- Softmax — converts a vector of scores into a probability distribution over classes.
- GELU / SiLU — smoother variants of ReLU used in many modern transformer architectures.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, 10),
)
```

Figure 2.1 — A simple feedforward network in PyTorch

2.2 Backpropagation and Gradient Descent

Training a network means finding the weights that minimize a loss function. Backpropagation computes the gradient of the loss with respect to every weight in the network using the chain rule, working backward from the output layer. An optimizer — most commonly Adam or a variant of it — then uses

those gradients to update the weights in small steps. This process repeats over many batches of data until the loss stops improving meaningfully.

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(epochs):
    for batch_x, batch_y in dataloader:
        optimizer.zero_grad()
        predictions = model(batch_x)
        loss = loss_fn(predictions, batch_y)
        loss.backward()
        optimizer.step()
```

Figure 2.2 — A standard PyTorch training loop

3. Key Architectures

3.1 Convolutional Neural Networks (CNNs)

CNNs are built around convolutional layers that slide small learnable filters across an image, detecting local patterns like edges in early layers and increasingly abstract shapes in deeper layers. Because the same filter is reused across the whole image, CNNs need far fewer parameters than a fully connected network would for the same input size, and they naturally generalize to patterns that appear anywhere in the image. Pooling layers downsample feature maps between convolutions to reduce computation and add a degree of translation invariance.

3.2 Recurrent Networks and Their Limits

Recurrent neural networks (RNNs) and their gated variants (LSTM, GRU) process sequences one element at a time, carrying a hidden state forward that summarizes everything seen so far. They were the standard approach to sequence modeling for years, but they are slow to train because processing is inherently sequential, and they struggle to retain information across very long sequences even with gating mechanisms.

3.3 The Transformer

The transformer architecture, introduced in the 2017 paper "Attention Is All You Need," replaced recurrence with self-attention: every position in a sequence attends directly to every other position, weighted by a learned relevance score. This makes training highly parallelizable across a whole sequence at once, and it captures long-range dependencies far more effectively than recurrence. Transformers are now the dominant architecture behind large language models, most modern vision models, and much of speech and multimodal AI.

```
attention(Q, K, V) = softmax(Q @ K.T / sqrt(d_k)) @ V
# Q, K, V are learned projections of the input sequence;
# the softmax term produces attention weights over all positions.
```

Figure 3.1 — Scaled dot-product self-attention

4. Training Deep Networks in Practice

Getting a large network to train well takes more than picking an architecture. A number of supporting techniques have become standard practice because they make optimization faster and more stable, and they reduce overfitting on models with very large parameter counts.

Technique	Purpose
Batch normalization	Stabilizes activations, allows higher learning rates
Dropout	Randomly disables neurons during training to reduce overfitting
Learning rate scheduling	Anneals the learning rate for smoother convergence
Data augmentation	Synthetically expands training data (crops, flips, noise)
Transfer learning	Fine-tunes a model pretrained on a large dataset
Mixed precision training	Uses lower-precision numbers to speed up training on GPUs

Table 4.1 — Common techniques for training deep networks

4.1 Transfer Learning

Training a large model from scratch is expensive, so in practice most projects start from a model already pretrained on a large, general dataset (e.g. ImageNet for vision, or a large text corpus for language) and fine-tune it on a smaller, task-specific dataset. This dramatically reduces both the data and compute needed to reach strong performance, and it is the standard approach in nearly every applied deep learning project today.

4.2 Hardware Considerations

Deep learning's computational cost comes from matrix multiplication at scale, which is why GPUs — and increasingly specialized accelerators like TPUs — are essential rather than optional. Training time scales with model size, dataset size, and number of epochs, and for the largest models, training is distributed across many GPUs or machines using techniques like data parallelism and model parallelism.

5. Where Deep Learning Is Applied

- Computer vision — object detection, medical image analysis, autonomous driving perception.
- Natural language processing — translation, summarization, and the large language models covered in the next guide.
- Speech — recognition, synthesis, and voice assistants.
- Recommendation systems at large scale, using deep embeddings of users and items.

- Scientific applications — protein structure prediction, drug discovery, weather forecasting.

6. Closing Thoughts

Deep learning's core ideas — layered nonlinear transformations trained end-to-end with backpropagation — are simple to state but have proven remarkably general. The shift from hand-designed features to learned representations, combined with the transformer's ability to scale, is the direct foundation for the generative AI systems covered in the next guide. Understanding these architectural and training fundamentals is what makes it possible to reason about why large models behave the way they do, rather than treating them as an unexaminable black box.